# GPTAid-Upgraded: Automated Generation of API Parameter Security Rules via Execution Feedback and Multi-Agent Refinement

**Stanley Chung**

**syc2153**

**Project Overview**

This project develops an enhanced GPTAid framework by forking the original codebase and implementing the proposed Bounded Inter-procedural Retrieval Layer and the Critic Agent. These components address the Lack of Context and the Incorrect Key Identification limitations mentioned in the original paper. The enhanced framework will be evaluated against the original one on precision, recall, bug detection rate, and average cost per API.

**How it works**

The GPTAid-Upgraded framework expands the original GPTAid by adding Bounded Inter-procedural Retrieval Layer at stage 1, and the Critic Agent at stage 3. The Retrieval Layer provides more source code and context when an API is calling another function within its body, and the Critic Agent provides a "final check" on the generated APSRs to ensure robustness. As a result, the full framework had an increase in precision from 50.0% to 57.9% compared to the baseline.

**Key Innovations**

- **Bounded Inter-procedural Retrieval Layer**

  This layer dynamically retrieves the code of external functions within APIs from the pre-downloaded code base. This procedure is bounded by a parameter $k \leq 2$ to avoid context explosion. The code base is pre-downloaded rather than dynamically retrieved online to isolate the layer and ensure our finding directly reflects the efficacy of the component.

- **Critic Agent**

  This agent acts as a "verifier" to check the quality of APSRs that enter the 3rd stage. The Critic agent takes the proposed rule and the correct and violation code as input, performs code difference analysis, and checks if the APSR matches the code difference. The APSR will be approved if it matches the code difference and discarded otherwise. A feedback loop is not implemented for invalid proposals due budget limitations

# Evaluation and Ablation Study

**LLM Model:** gemini-2.5-flash-lite

**Dataset:** The evaluation uses 20 **libpcap** APIs drawn from the same benchmark as the original GPTAid paper. libpcap is a widely-used C library for low-level network packet capture, and its APIs were chosen because they represent realistic functions with non-trivial parameter constraints. Due to budget constraints from LLM API costs, this work evaluates a subset of 20 APIs rather than the full benchmark, but the subset covers the same categories of constraints such as null-pointer requirements, activation ordering, and buffer sizing so results remain comparable to the original paper's findings. Furthermore, some APIs invoke other functions, making them good test cases for the Retrieval Layer.

**Metrics**

**Valid Violation** = The number of rules which the violation code does cause runtime error.

**Invalid Violation**: The generated violation code does not cause runtime error.

**Precision** = Final Rule Count/ (Final Rule Count + Invalid Violation).

**Final Rule Count**: APSRs surviving Stage 2 + the Critic Agent if the Critic Agent is active.

**Variants:**

- **Baseline**: The original framework. Both the Retrieval Layer and Critic Agent are deactivated.
- **Dynamic Only**: The original framework + Retrieval Laer , Critic Agent deactivated
- **Full**: The original framework + Retrieval Layer + Critic Agent

## Framework Performance

### The Critic Agent

It is observed that 33% of rules are dropped in all test cases where the Critic Agent is active. After manual review, the Critic Agent successfully acts as a “verifier” and drops rules which their description does not match the fundamental reason why the violation code is causing runtime error.

### The Retrieval Layer and the max_rules_per_api Variable

Tables 1-3 present an ablation study of the framework's precision across varying max_rules_per_api limits and Retrieval Layer configurations. While the Full variant includes both Retrieval and the Critic Agent, the Critic Agent is a post-generation filter and does not influence the initial discovery of valid violations. Therefore, for the scope of this section, the Full variant is simply an additional evaluation of the Retrieval mechanism. The data demonstrates that the Retrieval Layer does significantly increase precision except when max_rules_per_api=50. Across all three configurations, the Full variant achieves best precision at the lowest rule cap (74.9% at max=5), but precision decreases as the cap increases (67.4% at max=20, and 50.0% at max=50). This pattern suggests that the rules the LLM generates first are its most confident ones and tend to target clear, concrete constraints visible directly in the API source code. As the cap rises, the framework includes rules that are harder to validate or are just invalid, leading to more of their violation code failing at Stage 2. This observation leads to the conclusion that users should set a reasonable max_rules_per_api instead of letting LLMs generate as many raw rules as possible in stage 1.

| Variant | Valid Viol | Invalid Viol | Precision |
|---|---|---|---|
| Baseline | 26 | 14 | 65.0% |
| Dynamic Only (depth 1) | 24 | 11 | 68.6% |
| Full | 23 | 8 | 74.9% |

**Table 1. Component Ablation (max_rules_per_api=5)**

| Variant | Valid Viol | Invalid Viol | Precision |
|---|---|---|---|
| Baseline | 25 | 13 | 65.8% |
| Dynamic Only (depth 1) | 23 | 13 | 63.9% |
| Full | 29 | 14 | 67.4% |

**Table 2. Component Ablation (max_rules_per_api=20)**

| Variant | Valid Viol | Invalid Viol | Precision |
|---|---|---|---|
| Baseline | 28 | 18 | 60.9% |
| Dynamic Only (depth 1) | 29 | 22 | 56.7% |
| Full | 18 | 18 | 50.0% |

**Table 3. Component Ablation (max_rules_per_api=50)**

## The Retrieval Depth Study

Building on the precision improvements observed at Depth=1 in Tables 1–3, we examine whether performance continues to scale as retrieval depth increases to 2 by running a single evaluation on the **pcap_parsesrcstr_ex** API with max_rules_per_api=5. The 0 final rule count result shows that Depth=2 completely eliminates the framework's ability to generate rules with. This result is attributed to context overflow caused by the exponential expansion of injected code as each depth 1 function might invoke multiple others. As shown in Table 4., Depth=2 injected 341k tokens during stage 1, a 10.2x increase in token usage compared to baseline, and 4.72x more token used compared to Depth=1. This volume of data exhausts the LLM’s effective context window, preventing the generation of coherent output. This finding establishes that while one level of inter-procedural context is beneficial, deeper traversal causes a catastrophic collapse in performance.

| **Variant** | **Total token used** | **Final Rule Count** |
|---|---|---|
| **Baseline (depth=0)** | **31,222** | **4** |
| **Depth = 1** | **61,721** | **3** |
| **Depth = 2** | **349,408** | **0** |

**Table 4. Retrieval Depth vs. Tokens Used on the pcap_parsesrcstr_ex API**

# Answers to Research Questions

**RQ1: How does bounded inter-procedural retrieval affect the precision of initial APSR generation?**

The Bounded Inter-procedural Retrieval Layer improves the precision of APSR generation by providing the Large Language Model (LLM) with necessary source code and context when an API calls external functions within its body.

The impact on precision is characterized by the following findings:

- **Improvement at Depth=1:** At a retrieval depth of k = 1, the layer significantly increases precision across most test configurations. For example, when the rule generation cap is set to 5, the Full variant achieves **74.9%** and **68.6% precision**, an improvement over the **65.0% baseline**.
- **Sensitivity to Rule Volume:** The precision benefits are most pronounced at lower rule caps (e.g., max=5). As the **max_rules_per_api** increases to 50, the precision gains from the retrieval layer diminish.
- **The Context Explosion Boundary:** The retrieval must be strictly bounded ($k < 2$ in this case) to avoid overwhelming the model. While one level of context is beneficial, increasing the depth to k=2 results in a **catastrophic collapse** in performance. At this depth, the exponential expansion of injected source code from secondary function calls causes context overflow, preventing the framework from generating any valid rule.
- **Outcome:** When integrated into the full upgraded framework (including the Critic Agent), this retrieval layer contributes to an overall increase in precision from **50.0% to 57.9%** compared to the baseline.

**RQ2: To what extent does the Critic Agent reduce false positives from incorrect key operation identification?**

The Critic Agent reduces false positives by roughly 33% by acting as a secondary verification layer that filters out inaccurate rules before they are finalized. Its impact is defined by the following:

- **Verification Mechanism:** The agent performs a code difference analysis between correct and violation code to ensure the proposed security rule (APSR) accurately reflects why a runtime error occurred.
- **Filtering Rate:** In all test cases where the agent was active, it dropped approximately 33% of the proposed rules.
- **Qualitative Accuracy:** Manual reviews confirmed the agent successfully identified and discarded rules where the description did not match the fundamental technical reason for the code's failure.
- **Outcome:** By discarding these mismatched proposals, the agent directly addresses the “Incorrect Key Identification” limitation, ensuring that the final output is technically robust.

**RQ3: Which enhancement contributes more — the Retrieval Layer or the Critic Agent?**

The Retrieval Layer contributes more as it improves precision by proactively providing external context, while the Critic Agent takes a more passive approach by trimming the final result. However, the Retrieval Layer's effectiveness is strictly dependent on the retrieval depth, described by the following:

- **Context Enhancement**: The layer retrieves the source code of external functions called within an API's body to resolve the "Lack of Context" limitation.
- **Depth=1 Performance**: At a traversal depth of one, the layer increases precision from **65.0%** to **68.6%** and **74.9%**, helping the LLM understand complex parameter constraints.
- **Depth=2 Collapse**: Increasing depth to two levels causes a catastrophic failure, resulting in zero rules generated due to an exponential expansion of injected code.
- **Resource Consumption**: Depth=2 triggers a 10.2x increase in token usage compared to the baseline, which exhausts the LLM's effective context window and prevents coherent output.
- **Outcome**: This component establishes a clear functional boundary where a single level of inter-procedural context is beneficial, but deeper traversal becomes counterproductive.

# Deliverables and Documentation

**Repository Link:** https://github.com/aldStanley/GPTAid_expanded

For information about reproducing the result, please refer to README in the project Github.

**External Materials:**

- **Original Paper:** https://arxiv.org/abs/2409.09288
- **Original Repo:** https://github.com/icy17/GPTAid/?tab=readme-ov-file

# Future Work & Research Direction

While the current results demonstrate an increase in APSR precision, the scope of this project is bounded by the constraints of a single-semester research cycle. Future efforts to refine GPTAid-Upgraded will focus on the following aspects:

- **Comprehensive Benchmarking:** Due to budget constraints regarding LLM token usage and limited access to the original study's full benchmark set, the current evaluation was restricted to a subset of the **libpcap** library. A primary objective for future work is to conduct a head-to-head comparison across the entire libpcap dataset to validate the precision gains against the result in the original paper

- **Optimizing Retrieval Depth:** Given that depth=2 retrieval currently yields a 0% rule generation rate due to context explosion, it is suggested to conduct research on Selective Context Summarization to allow the model to gain deeper inter-procedural logic without exceeding token limits.

# Findings and Lessons Learned

One clear finding from the project result is that more context does not always mean better results. Adding depth 1 inter-procedural retrieval can improve precision by up to 10%, but increasing depth to 2 overloads the framework with too much context and leads to 0 final rule. In addition, I learned that it is very difficult to automate natural language output evaluation in the traditional way because sentences that have completely different structure can still have the same meaning, making it difficult to define one answer as the ground truth. For example, for the source code **if(A and B and C) …**, "parameter A, B, and C must be either true or false" and "the condition must evaluate to either True or False" are technically the same constraint, but different users may prefer one over the other. This challenge demonstrates the importance of the Critic Agent. By shifting the focus from linguistic matching to code difference analysis and execution feedback, the framework can verify the functional impact of a rule even when its natural language description varies significantly in structure.

Another lesson learned during the evaluation process is the necessity of human-in-the-loop verification on the LLM output. During precision evaluation, I noticed misalignment in the data the LLM fetched, and later realized that Claude Code misinterpreted the meaning of the data and cascaded to a hallucinated conclusion. This experience shows that even though agentic tools can speed up the development process at a high task completion rate, hallucination is still a high stake issue to be solved especially as agentic tools are handling more and more complicated tasks. One small mistake overlooked in the early step can lead to cascading hallucination as the process builds up, and results in something far from what is expected. Thus human verification still plays an important part in the AI era particularly when it comes to high stake tasks.

# Self-Evaluation

- **Accomplishments:** Successful GPTAid-Upgraded implementation that outperforms the original framework.
- **Teamwork Contribution:** The project was developed independently using Claude Code, with no help from any other external sources.

# Appendices

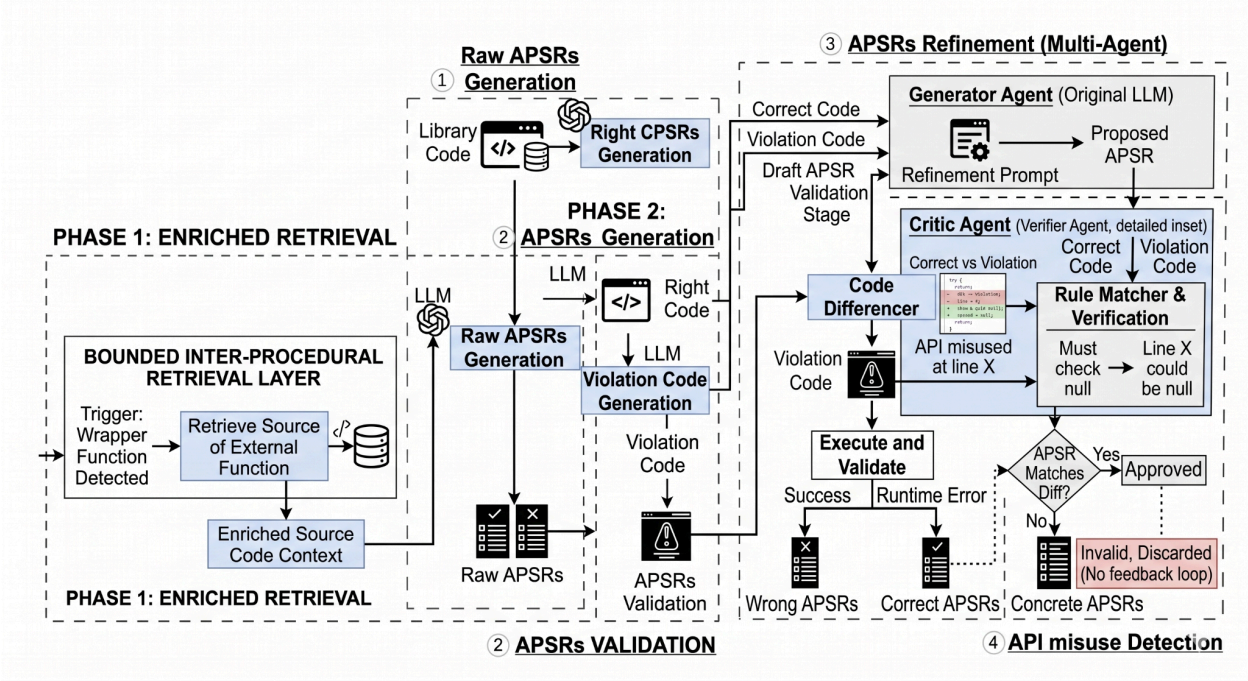


**Figure 1. GPTAid-Upgraded Framework Diagram**